Exploring and Testing APIs with Postman

Name: Kalungi Bright Angel
GitHub Repository link: https://github.com/Kalungibrightangel/working_with_APIs_1

1. Introduction

This project was completed using Postman to explore and test an API.
The API selected for this project was DummyJSON. DummyJSON provides different API resources that can be accessed using HTTP requests. It also provides authentication, which made it suitable for me to demonstrate authentication, CRUD operations, environment variables, automated testing, and the Collection Runner.

API Name: DummyJSON

Base URL: https://dummyjson.com

DummyJSON was selected because it provides endpoints for authentication and product management.

The project focused mainly on the Products API.

The main endpoints used were:

Purpose	Method	Endpoint
Login	POST	/auth/login
Get Products	GET	/products
Create Product	POST	/products/add
Update Product	PUT	/products/add/resource_id
Delete Product	DELETE	/products/add/resource_id

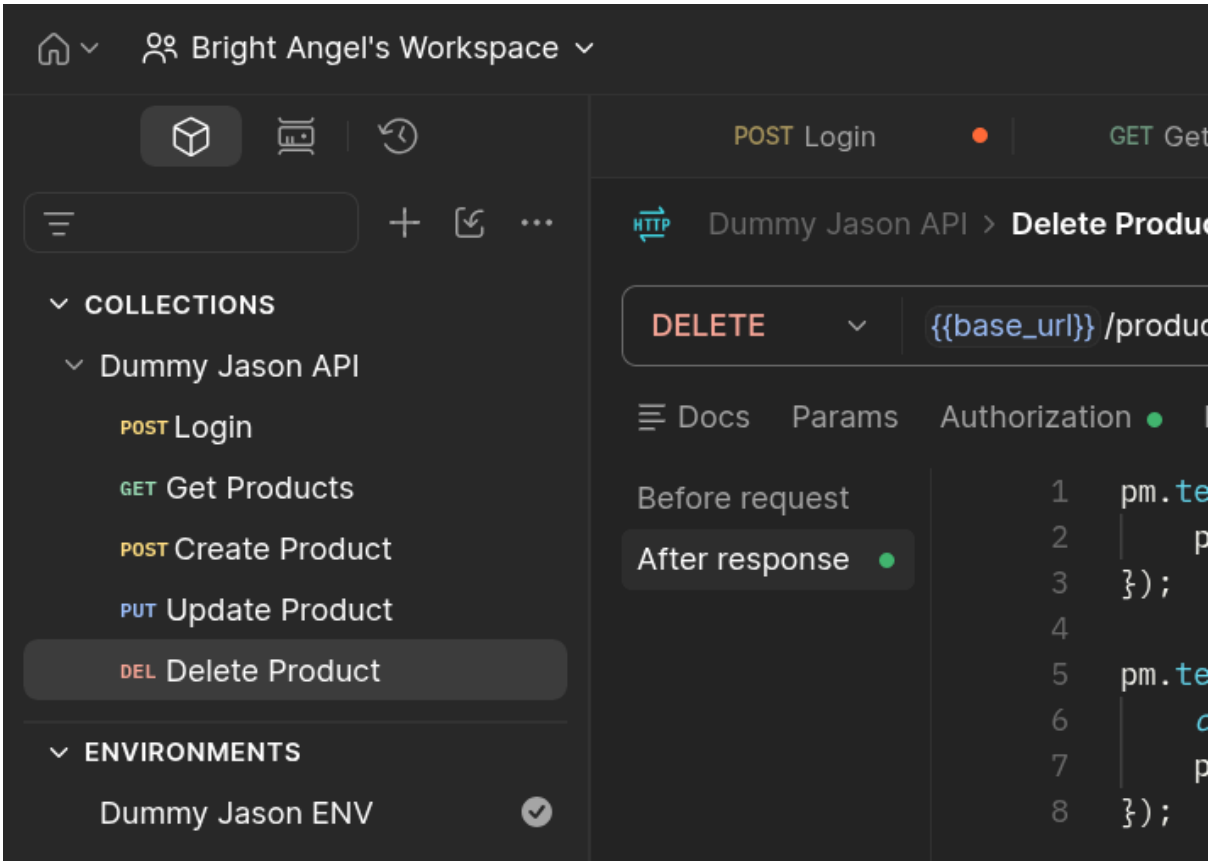
These endpoints allowed me to demonstrate the main HTTP methods required by the assignment.

2. Postman Collection
A Postman collection named DummyJSON API Assignment was created to organize all the requests.

The collection contains five requests:

01 - Login
02 - Get Products
03 - Create Product
04 - Update Product
05 - Delete Product

Screenshot



3. Authentication

Authentication was implemented using the DummyJSON login endpoint.

Method: POST

URL: {{base_url}}/auth/login
Instead of writing the API URL directly, the login request uses the base_url environment variable.

The request body contains a username and password.

```
{
  "username": "emilys",
```

```
  "password": "emilyspass"
}
```

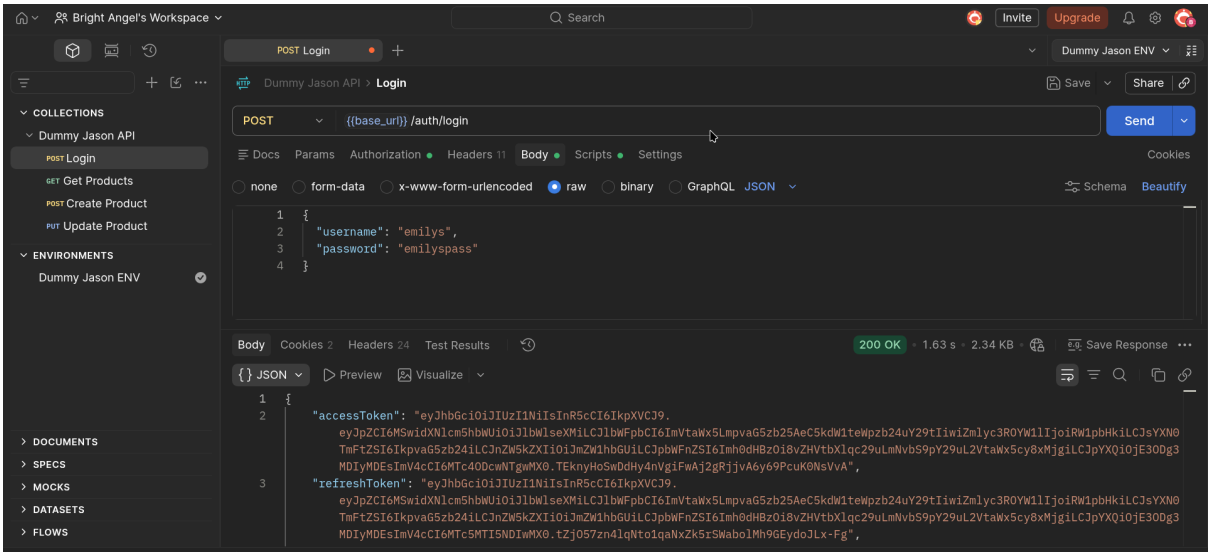
When the request is successful, the API returns an access token.

The token is then stored in the Postman environment using a Post-response script.

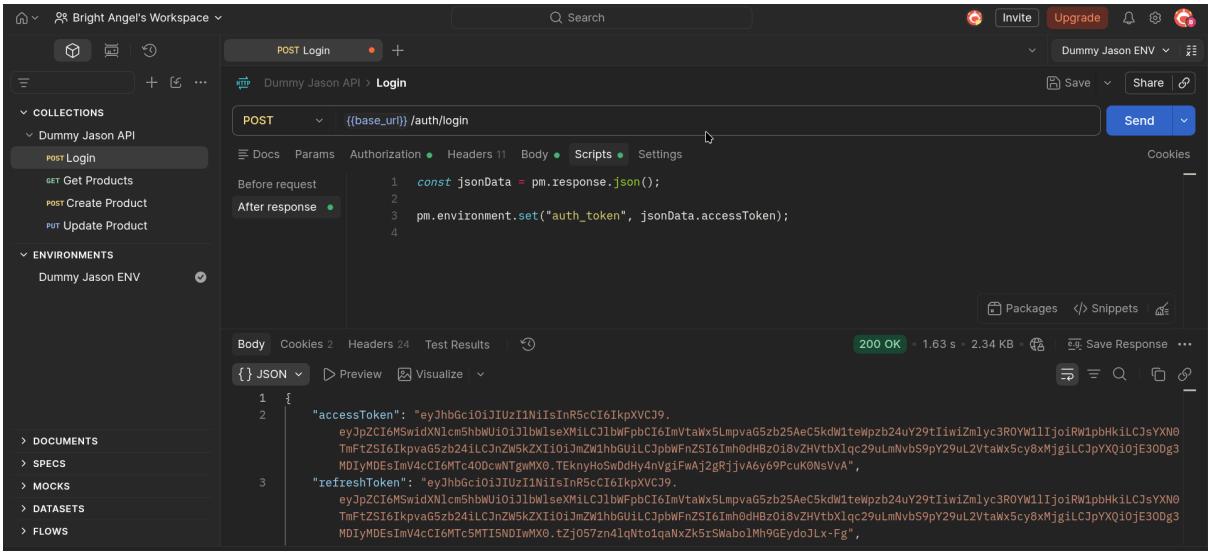
```
const jsonData = pm.response.json();

pm.environment.set("auth_token", jsonData.accessToken);
```

This allows the token to be reused by other requests without manually copying and pasting it.

Screenshots
Successful Login



Script that stores the token into the environment:



4. Environment Variables

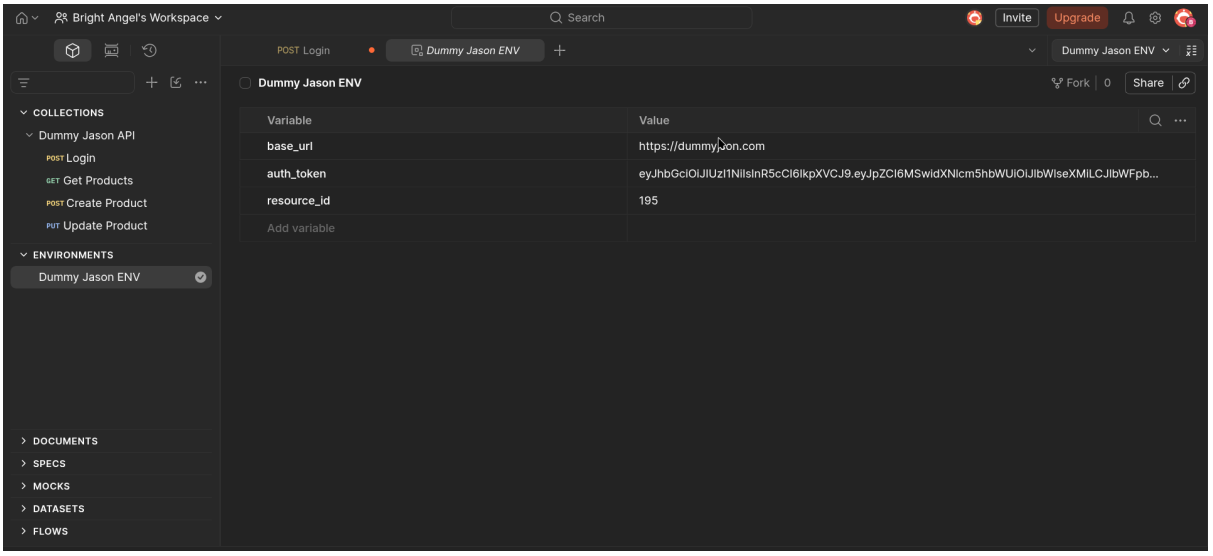
A Postman environment named DummyJSON Environment was created.

The environment contains the following variables:

Variable	Purpose
base_url	Stores the API base URL
auth_token	Stores the authentication token
resource_id	Stores the product ID used by PUT and DELETE

The environment makes the requests easier to manage because values can be reused instead of being written manually in every request.

Screenshot 4: Environment Variables



5. GET Request: Retrieve Products

The first CRUD operation tested was GET.

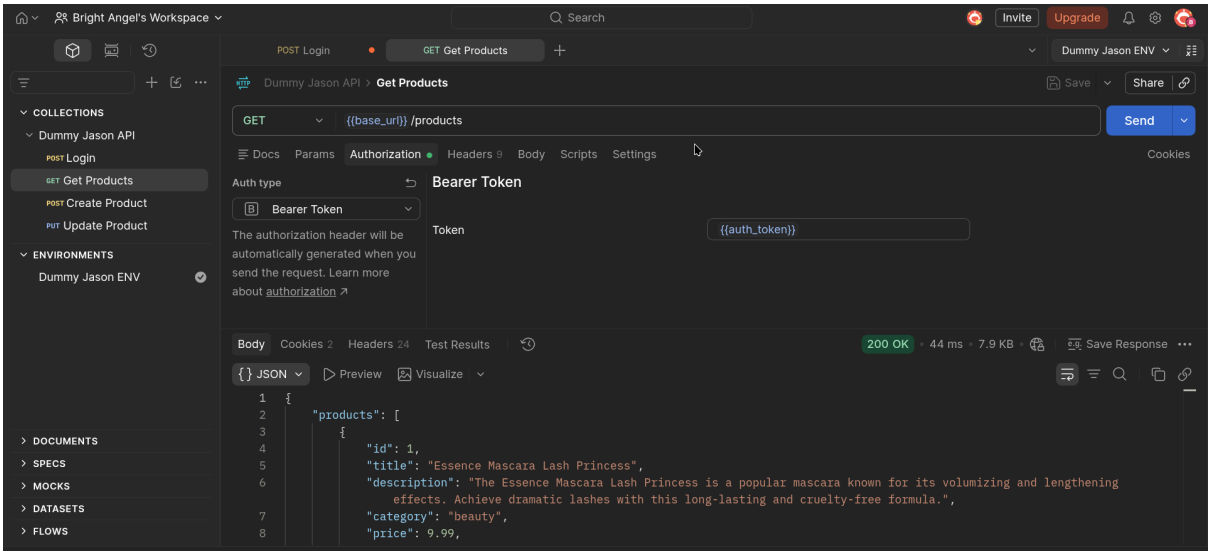
Method: GET

Endpoint: {{base_url}}/products

The request uses the stored authentication token with Bearer Token authentication.

The API returned a list of products successfully.

Screenshot : GET Products



6. POST Request Create Product

The POST method was used to demonstrate creating a product.

Method: POST

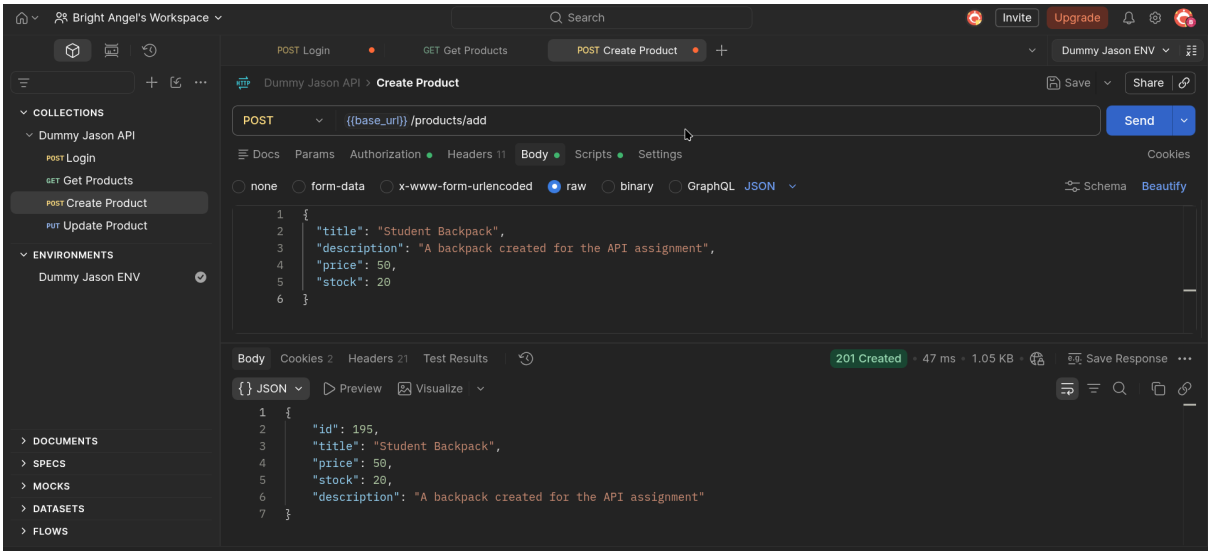
Endpoint: {{base_url}}/products/add

The request body was:

```
{
  "title": "Student Backpack",
  "description": "A backpack created for the API assignment",
  "price": 50,
  "stock": 20
}
```

The API returned a response containing the created product information and an ID.

Screenshot: Create Product



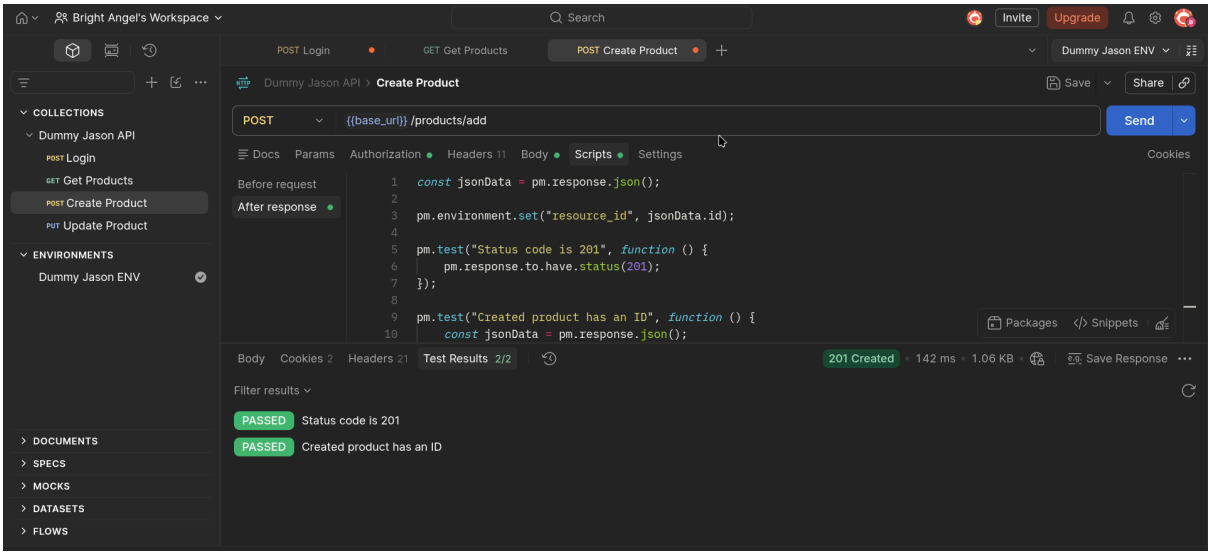
Scripts:
The first script adds the product ID to the environmental environment, under resource_id.

```
const jsonData = pm.response.json();

pm.environment.set("resource_id", jsonData.id);
```

Tests were also added to verify that the request returned status code 200 and that the response contained a product ID.

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Created product has an ID", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.id).to.exist;
});
```

Screenshot 15: POST Tests



7. PUT Request: Update Product

The PUT method was used to update an existing product.

Method: PUT

Endpoint: {{base_url}}/products/{{resource_id}}

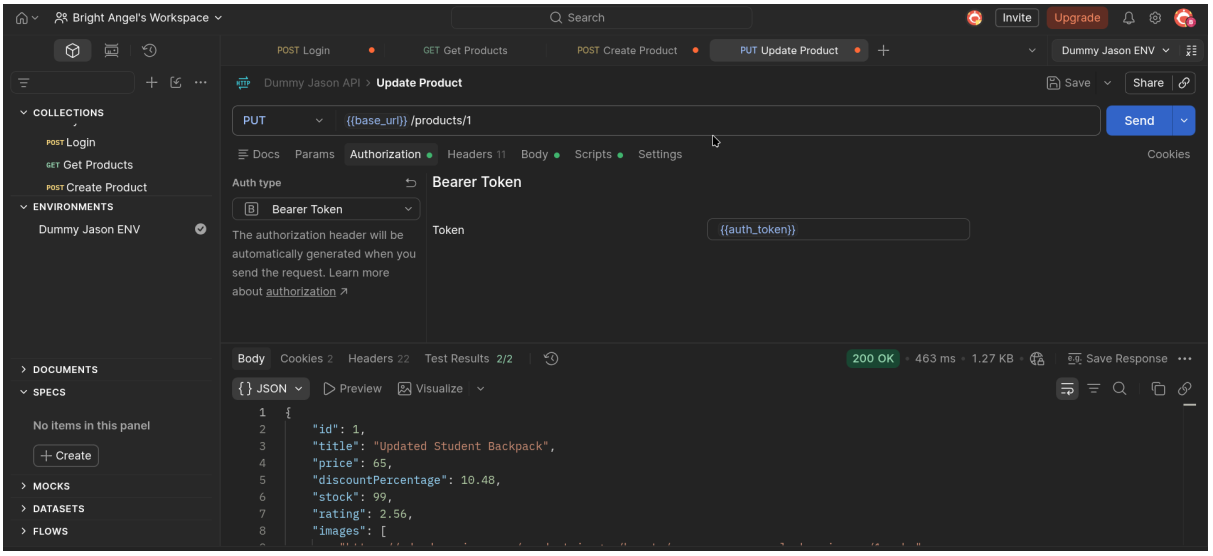
The resource_id variable was set to 1.

The request body was:

```
{
  "title": "Updated Student Backpack",
  "price": 65
}
```

The API returned the updated product information successfully.

Screenshot: PUT Request



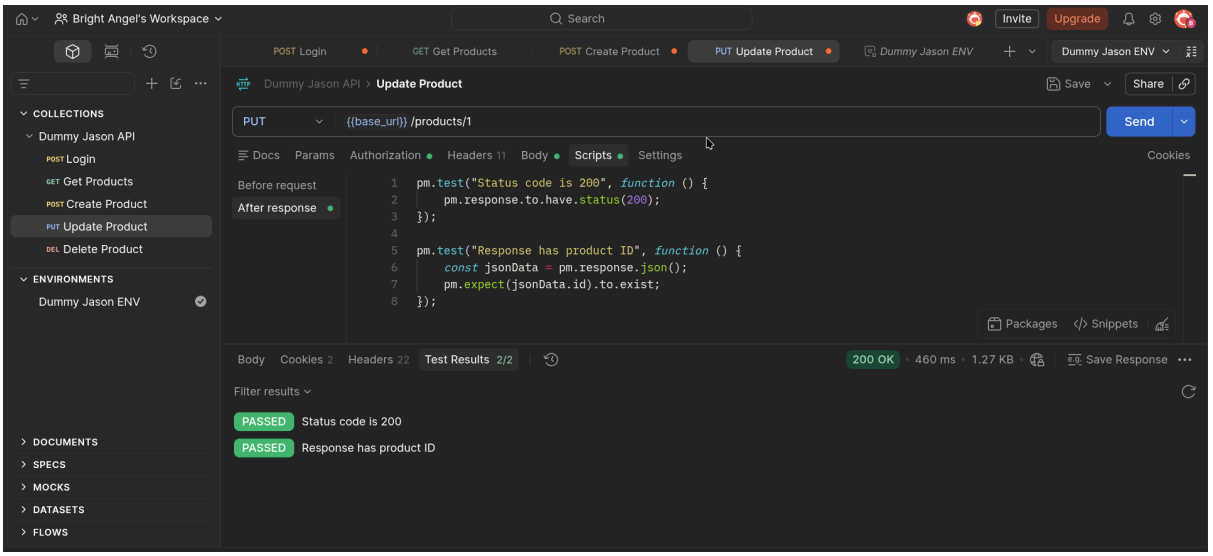
PUT Tests

Two tests were used:

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response has product ID", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.id).to.exist;
});
```

Both tests passed successfully.

Screenshot: PUT Tests



8. DELETE Request: Delete Product

The DELETE method was used to demonstrate deleting a product.

Method: DELETE

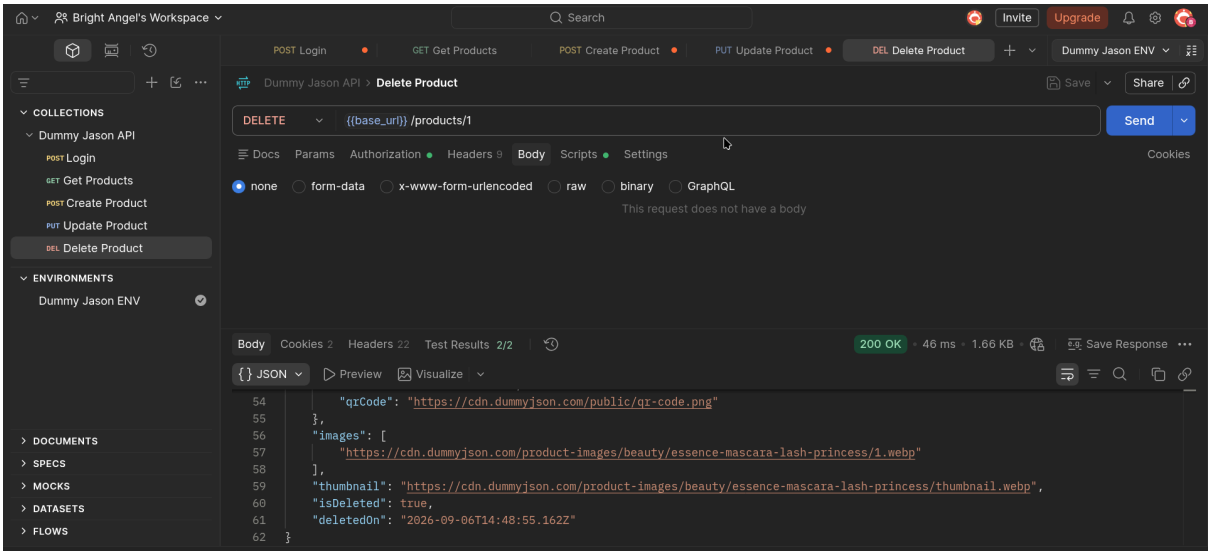
Endpoint: {{base_url}}/products/{{resource_id}}

The resource_id variable was set to 1.

No request body was required.

The API returned a successful deletion response.

Screenshot: DELETE Request



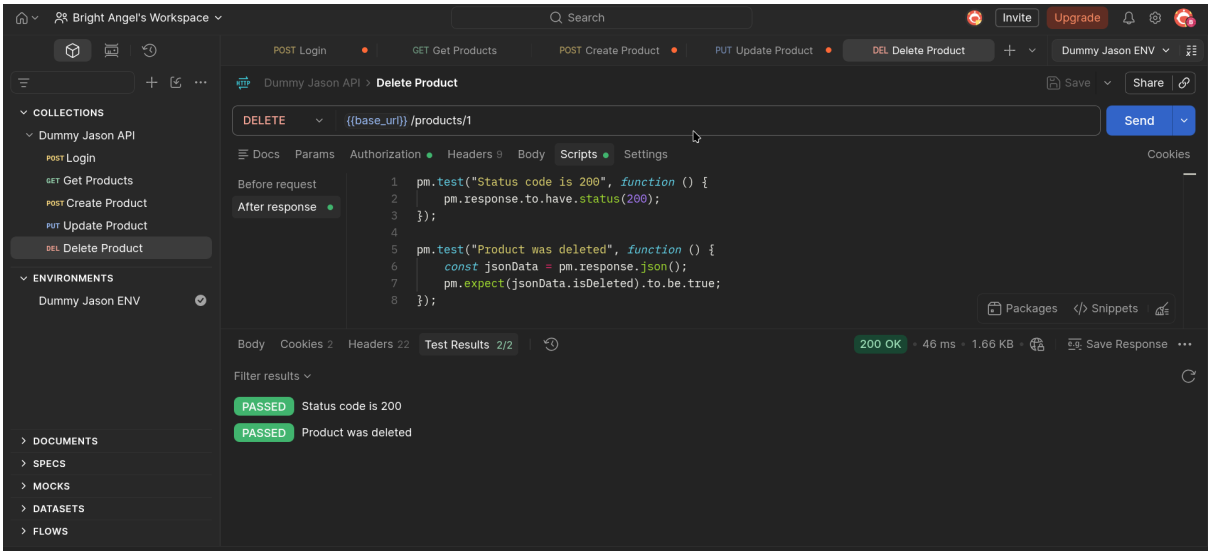
DELETE Tests

The following tests were used:

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Product was deleted", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.isDeleted).to.be.true;
});
```

Both tests passed successfully.

Screenshot: DELETE Tests



12. Pre-request Script and Token Automation

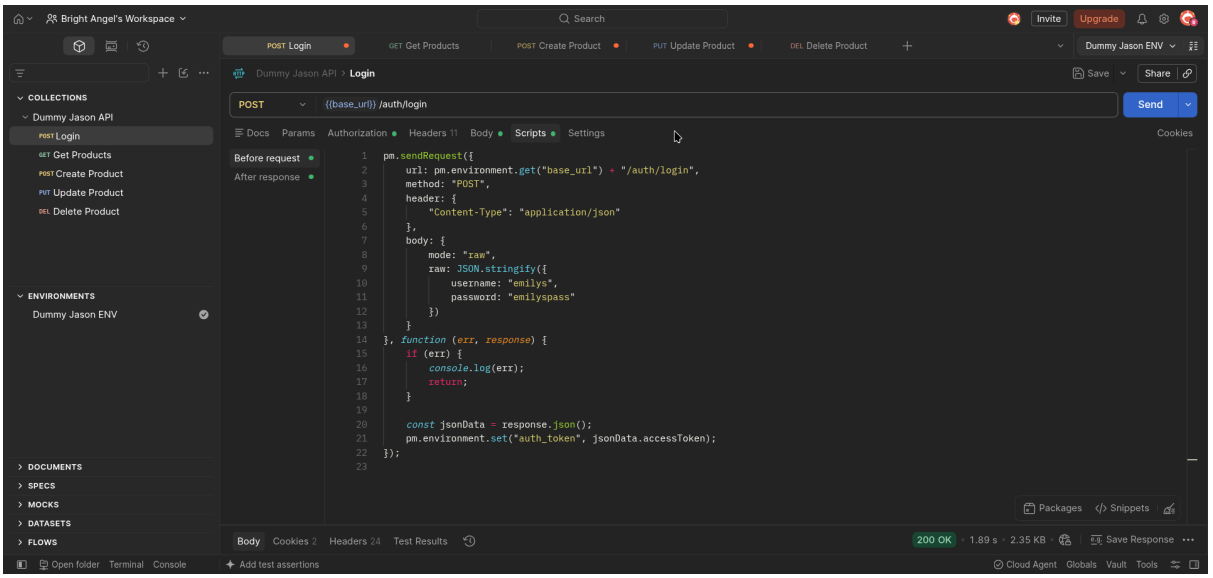
A Pre-request Script was also used to demonstrate automatic authentication before a protected request.

The script sends a login request and saves the returned access token into the environment.

```
pm.sendRequest({
    url: pm.environment.get("base_url") + "/auth/login",
    method: "POST",
    header: {
        "Content-Type": "application/json"
    },
    body: {
        mode: "raw",
        raw: JSON.stringify({
            username: "emilys",
            password: "emilyspass"
        })
    }
}, function (err, response) {
    if (err) {
        console.log(err);
        return;
```

```
    }

    const jsonData = response.json();
    pm.environment.set("auth_token", jsonData.accessToken);
});
```

This demonstrates how Postman can automatically obtain and store a token before sending a protected API request.
Screenshot:



9. Collection Runner and Automation

The Postman Collection Runner was used to execute multiple requests automatically.

The collection was configured with:

Collection: DummyJSON API Assignment
Environment: DummyJSON Environment
Iterations: 1

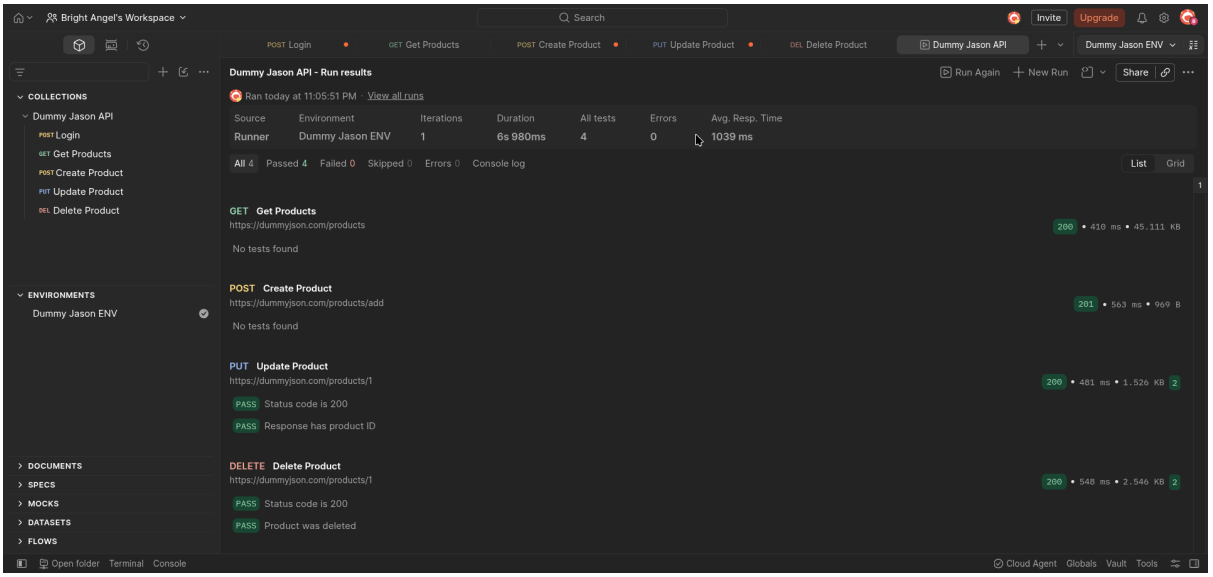
The five requests were executed as part of the automated workflow:

Login
Get Products
Create Product
Update Product
Delete Product

The requests used environment variables such as {{base_url}}, {{auth_token}}, and {{resource_id}}.

The tests were also executed automatically during the collection run.

Screenshot 16: Collection Runner



API Testing Results

The API requests were tested successfully using Postman.

The tests checked HTTP status codes and important response values such as product IDs and the isDeleted property.

Important Notes About DummyJSON

DummyJSON simulates some product modification operations. Therefore, the POST, PUT, and DELETE operations are useful for demonstrating API requests and responses, but the changes are not treated like permanent changes to a production database.

For the PUT and DELETE demonstrations, product ID 1 was used because it is an existing product resource.

This was also why the environment variable resource_id was kept as 1 rather than relying on the ID returned by the simulated POST operation.

Challenges Encountered

During the project, several challenges were encountered.

1. Resource ID

The POST operation returned a product ID, but using that ID for later requests could result in a 404 response because DummyJSON's product creation is simulated.

The solution was to use the existing product ID 1 for the PUT and DELETE demonstrations.

2. Collection Runner

Some requests behaved differently when the complete collection was executed automatically. The requests were therefore tested individually to identify the issue and verify that each CRUD operation worked correctly.

Conclusion

This project provided practical experience with using APIs through Postman.

I learned how to:

Select and explore an API.
Authenticate using an API login endpoint.
Send GET, POST, PUT, and DELETE requests.
Use Bearer Token authentication.
Create and use Postman environment variables.
Automatically store authentication tokens.
Write Postman test scripts.
Use Pre-request Scripts.
Run multiple API requests automatically using the Collection Runner.
Identify and troubleshoot API errors such as 404 responses.

Overall, the project improved my understanding of how APIs communicate with applications and how Postman can be used to test and automate API workflows.